Integrative Programming II

Student Manual — Lessons 9 to 12: From Frontend to Full Stack

Before You Start

In your earlier lessons, you built a React Todo App that saves tasks in your browser using `localStorage`. That worked, but it has one big problem: the data lives only on your computer, in your browser. If you open the app on another device, or clear your browser data, everything disappears.

Starting with Lesson 9, you will build a **backend** — a small server program that stores your todos and lets your React app talk to it over the internet. By the end of Lesson 12, your Todo App will be a real full-stack application.

New words you will meet in these lessons: Backend, Server, Express, Middleware, CORS, Port, JSON, REST API, Endpoint, Postman. Don't worry — each one will be explained the first time it appears.

Lesson 9: Backend Project Setup

1. Objective

Right now, your project folder only has your React app (the frontend). In this lesson, you will create a **second, separate folder** for your backend. Think of it as building a new "kitchen" that will prepare and serve data to your React app's "dining room."

By the end of this lesson, you will have:

- A new project folder named `todo-backend`
- A working Node.js project inside it
- Two important packages installed: `express` and `cors`

2. Steps

Step 1 — Create the backend folder

Open your terminal. Navigate to the same parent folder where your React Todo App lives (not inside it — beside it).

```
mkdir todo-backend
cd todo-backend
```

`mkdir` means "make directory." You just created a brand-new, empty folder called `todo-backend`, and then moved into it with `cd` (change directory).

Step 2 — Turn the folder into a Node.js project

```
npm init -y
```

This command creates a file called `package.json`. This file is like an ID card for your project — it lists your project's name, version, and (soon) the packages it depends on. The `-y` flag means "yes to all the default questions," so it happens instantly without asking you anything.

Step 3 — Install Express and CORS

```
npm install express cors
```

- **Express** is a toolkit that makes it much easier to build a server in Node.js. Without it, you would have to write a lot of complicated code just to handle web requests.
- **CORS** (Cross-Origin Resource Sharing) is a small helper that gives your React app permission to talk to your backend, even though they run on different addresses (ports).

This command also creates:

- A `node_modules` folder (contains the actual code for Express, CORS, and their helpers)
- A `package-lock.json` file (locks the exact versions used, so your project behaves the same on any computer)

3. Checkpoint

✅ Run this command and check your results:

```
ls
```

You should see:

```
node_modules/
package.json
package-lock.json
```

✅ Open `package.json` in your code editor. Inside, you should see a "dependencies" section listing `express` and `cors`, similar to:

```
"dependencies": {
  "cors": "^2.8.5",
  "express": "^4.19.2"
}
```

If you see this, your setup is correct! You are ready for Lesson 10.

4. Reflection Questions

Answer these in your own words on your yellow pad:

1. Why do we create the backend in a separate folder from our React app, instead of putting it inside the same folder?

2. What is the purpose of `package.json`? What would happen if you deleted it?

3. In your own words, what do you think a "server" does for an application?

Lesson 10: Express Server Setup

1. Objective

A folder with installed packages is not yet a server — it's just ingredients waiting to be cooked. In this lesson, you will write the actual code that starts a working server: `server.js`. When this lesson is done, your computer will be "listening" for requests, ready to serve data.

2. Steps

Step 1 — Create the server file

Inside your `todo-backend` folder, create a new file named `server.js`.

Step 2 — Import the tools you installed

At the very top of `server.js`, write:

```
const express = require("express");
const cors = require("cors");

const app = express();
```

- `require()` is how Node.js loads a package you installed. Think of it as "bringing the tool out of the toolbox."
- app is your actual server object. Every route and setting you add from now on will be attached to app.

Step 3 — Add your middleware

Right below, add:

```
app.use(cors());
app.use(express.json());
```

These two lines are called **middleware** — code that runs on every request, before it reaches your actual routes.

- `cors()` — allows your React app (running on a different port) to send requests to this server without being blocked.
- `express.json()` — automatically reads incoming JSON data and makes it available as a normal JavaScript object, so you don't have to decode it yourself.

Step 4 — Set the port and start listening

At the bottom of the file, add:

```
const PORT = 3000;

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

A **port** is like a numbered door on your computer. Your server will "stand" at door number 3000, waiting for visitors (requests).

Step 5 — Run your server

```
node server.js
```

3. Checkpoint

✅ Your terminal should print:

```
Server is running on http://localhost:3000
```

✅ Open a web browser and visit `http://localhost:3000`. You will likely see a message like "Cannot GET /" — this is actually good news! It means your server is alive and responding; you just haven't created any routes yet. That's what Lesson 11 is for.

✅ Press `Ctrl + C` in the terminal if you need to stop the server.

4. Reflection Questions

1. What is the difference between `require("express")` and `app.use(express.json())`? Why do we need both?

2. In your own words, explain what a "port number" is, using an analogy (like a door, a phone extension, etc.).

3. Why do we need the `cors()` middleware? What do you think would happen if we removed it?

Lesson 11: Building REST API Endpoints

1. Objective

Your server is running, but it has nothing to say yet. In this lesson, you will teach it how to store todos in memory and create two endpoints — specific addresses that your React app can visit to get data or send new data.

By the end, your server will support:

- `GET /api/todos` → "Give me the list of todos"
- `POST /api/todos` → "Here is a new todo, please save it"

2. Steps

Step 1 — Create your in-memory data

Below your middleware (`app.use(...)` lines) and above `app.listen`, add:

```
let todos = [
  { id: 1, text: "Learn Express", completed: false },
  { id: 2, text: "Build a REST API", completed: false }
];
```

This is a normal JavaScript array acting as a temporary database. It is called **in-memory** because it only exists while the server is running — if you stop the server, the data resets. (In later courses, you will replace this with a real database.)

Step 2 — Create the GET endpoint

```
app.get("/api/todos", (req, res) => {
  res.json(todos);
});
```

This means: "Whenever someone visits `/api/todos` using a GET request, send back the entire `todos` array as JSON."

- `req` (request) — information coming in from whoever is asking.
- `res` (response) — what you send back to them.
- `res.json(...)` automatically converts your JavaScript array into JSON format and sends it.

Step 3 — Create the POST endpoint

```
app.post("/api/todos", (req, res) => {
  const newTodo = {
    id: Date.now(),
    text: req.body.text,
    completed: false
  };
```

```
  todos.push(newTodo);
  res.json(newTodo);
});
```

This means: "Whenever someone sends a POST request to `/api/todos` with a new todo's text, create a new todo object, add it to the array, and send that new todo back as confirmation."

- `req.body.text` — this is only possible because of `express.json()` from Lesson 10! It reads the text sent by the client.
- `Date.now()` — generates a unique number based on the current time, used as a simple ID.
- `todos.push(newTodo)` — adds the new todo to your in-memory array.

3. Checkpoint

✅ Restart your server:

```
node server.js
```

✅ Visit `http://localhost:3000/api/todos` in your browser. You should now see JSON output like:

```
[
  { "id": 1, "text": "Learn Express", "completed": false },
  { "id": 2, "text": "Build a REST API", "completed": false }
]
```

✅ You cannot easily test POST in a browser address bar — that's exactly what Lesson 12 (Postman) is for!

4. Reflection Questions

1. What does "in-memory" data mean, and what is its biggest limitation?

2. Explain, in your own words, the difference between a GET request and a POST request.

3. Where does `req.body.text` come from, and which line of code from Lesson 10 makes it possible to read it?

Lesson 12: API Testing with Postman

1. Objective

A browser can only easily send GET requests. To properly test your POST endpoint — and to check your work like a professional developer — you will use Postman, a tool for sending any type of request to your server and inspecting the response.

2. Steps

Step 1 — Open Postman and make sure your server is running

In one terminal window, keep your server running:

```
node server.js
```

Open the Postman application (or the Postman web version).

Step 2 — Test your GET endpoint

1. Create a New Request.
2. Set the method dropdown to GET.
3. Enter the URL: `http://localhost:3000/api/todos`
4. Click Send.

You should see your todos array appear in the response panel at the bottom, along with a status code of 200 OK.

Step 3 — Test your POST endpoint

1. Create another New Request.
2. Set the method dropdown to POST.
3. Enter the URL: `http://localhost:3000/api/todos`
4. Click the Body tab.
5. Select raw, then choose JSON from the format dropdown on the right.
6. Type the following into the body box (shown below).
7. Click Send.

```
{
  "text": "Test my backend with Postman"
}
```

Step 4 — Confirm it worked

1. Look at the response panel — you should see your new todo returned, complete with an id and completed: false.
2. Send another GET request (Step 2) to confirm your new todo is now permanently part of the list (until the server restarts).

3. Checkpoint

✅ Your POST response should look similar to:

```
{
  "id": 1723456789012,
  "text": "Test my backend with Postman",
  "completed": false
}
```

✅ Your follow-up GET request should show three todos in the array (the original two plus your new one).

✅ If you get an error instead, check:

- Is node `server.js` still running in your terminal?
- Did you select raw → JSON in the Body tab (not "form-data" or "text")?
- Did you spell `text` correctly, matching `req.body.text` in your code?

4. Reflection Questions

1. Why can't we easily test a POST request just by typing a URL into a browser's address bar?

2. What is the purpose of choosing "raw" and "JSON" in Postman's Body tab before sending a POST request?

3. Describe, step by step, what happens from the moment you click "Send" in Postman until you see the new todo appear in the response.

Looking Ahead

Congratulations — you now have a real backend server with a working REST API! In upcoming lessons, you will connect this backend to your React frontend, replacing `localStorage` with real network requests using `fetch`.